main 1 Branch 0 Tags Go to file Code

Sebastian Wild BUGFIX: extension to minRunLen didn't use existing run 87ef163 · 3 weeks ago 9 Commits

src	BUGFIX: extension to minRunLen didn't use existing run	3 weeks ago
.gitignore	Initial commit	2 years ago
CMakeLists.txt	code for multiway powersort paper	2 years ago
LICENSE	Initial commit	2 years ago
README.md	formatting of readme	2 years ago
create-plots.nb	code for multiway powersort paper	2 years ago
experiments.sh	add experiment with varying presortedness	last year

README MIT license

Accompanying code for paper “Multiway Powersort”

This is the accompanying C++ code for the article "Multiway Powersort" by William Cawley Gelling, Markus Nebel, Benjamin Smith and Sebastian Wild. It contains efficient implementations of nearly-optimal adaptive mergesort variants, in particular Powersort and Peeksort as well as variants thereof for 4-way merging.

To reproduce the raw data used in the running-time study in the paper, run the `experiments.sh` Bash script. The code assumes a typical Unix development environment and uses `cmake`. The code is optimized for compilation with `g++` (from the GNU Compiler Collection), but should also work with `clang++` (from LLVM).

The cachegrind cache simulations need `valgrind` and `callgrind` to be installed.

The largest inputs (100M 16-byte objects) use up to 10 GB of main memory (buffers are not shared across algorithms); comment these runs out in `experiments.sh` if you do not have sufficient memory. The full set of experiments runs for ~1 day.

Figures

All figures in the paper are created using Wolfram Mathematica; the code is in the notebook `create-plots.nb`.

A quick description on how to manually recreate the plots when Mathematica is not an option appears below.

Figure 3 (running time over n, ints, random runs)

Uses the averages and standard deviations from `times-runs-int.out`.

Figure 4 (running time distribution, 10^7 ints, random runs)

Uses the raw data from `times-runs-10m-int-dist-xxxxx.csv` (where `xxxxx` is the timestamp of the run and a parameter summary). The charts are created using Mathematica's `DistributionChart` command.

Figure 5 (running time over n, records, random runs)

As Fig. 3, but with the data from `times-runs-l+p.out`.

Figure 6 (running time over n, ints, random permutations)

As Fig. 3, but but with the data from `times-rp-int.out`.

Figure 7 (scanned elements over n, random runs)

Uses the columns `merge-cost` and `buffer-cost` from `times-runs-*-cmps.csv` files; the scanned element count is computed as:

scanned elements = 2 * merge-cost + 2 * buffer-cost

Then the averages and standard deviations are plotted.

Figure 8 (merge cost over n, random runs)

Like Fig. 7, but using only column `merge-cost`.

Table 1 (cachegrind results)

The numbers in Table 1 can be read off using `cg_annotate` or `KCachegrind`.

List of implemented sorting algorithms

All algorithms are implemented as a C++ template. Apart from the iterator type, most have further parameters to fine-tune the algorithm.

- `powersort.h`: standard 2-way powersort implementation as described in Munro & Wild ESA 2018. Important parameters are the minimal run length (with shorter runs filled up to that size using Insertionsort) and the merge method.
- `powersort_4way.h`: 4-way powersort implementation as described in the paper. Parameters are as for powersort.
- `top_down_mergesort.h`: simple top-down mergesort, by default using Insertionsort on subproblems with <= 24 elements and skipping a merge when the two runs are already in order.
- `bottom_up_mergesort.h`: a simple bottom-up-mergesort, parameters as for top-down.
- `quicksort.h`: a relatively simple quicksort implementation, like GNU STL `std::sort`, but without the introsort part.
- `timsort.h`: C++ port of Timsort from https://github.com/timsort/cpp-TimSort/
- `trotsort.h`: Simplified version of `timsort.h` without galloping merge.
- `peeksort.h`: Peeksort as described in Munro & Wild ESA 2018.

About

C++ implementations of Powersort and 4-way Powersort

Readme
MIT license
Activity
12 stars
2 watching
0 forks
Report repository

Releases

No releases published

Packages

No packages published

Languages

- ● C++ 78.2%
- ● Python 10.0%
- ● Mathematica 6.6%
- ● Makefile 1.8%
- ● CMake 1.1%
- ● Shell 1.0%
- ● Other 1.3%